Lehrerexemplar · Boss-Kampf 1 · Welt 1

Erwartungshorizont: Die Gildenhalle – das Auftragsbrett

Erwartungshorizont / Bewertungsbogen für _____

I, II, III: erreichbare Bewertungseinheiten im jeweiligen Anforderungsbereich · BE: vom Prüfling erreichte Bewertungseinheiten

Erwartete Prüfungsleistungen	I	II	III	BE
a) Erläuterung des Aufbaus der Klasse Auftrag Kernpunkte: Beschreibung der Bestandteile Attribut, Konstruktor, Methode mit Bezug zur Klasse Auftrag; Bedeutung von - (nur innerhalb der Klasse) und + (auch von außen); Konstruktoren (c) erzeugen Objekte und belegen die Attribute; zwei Konstruktoren mit gleichem Namen, aber unterschiedlicher Parameterliste (Überladen), Standardwerte beim Konstruktor mit einem Parameter.	3	1		
b) Darstellung des Zustands mit Objektkarten; Angabe und Begründung Kernpunkte: Drei Auftrag-Objekte mit korrekten Attributwerten (Standardwerte 10/1 beim ersten); Felder 0 und 2 verweisen auf dasselbe Objekt, Feld 1 auf „Drachenei bergen“, Feld 3 null, anzahl = 3; a1 und a3 verweisen auf dasselbe Objekt, a2 auf das neue Objekt „Kräuter sammeln“; der Aufruf liefert wahr, da Feld 2 auf das über a3 erledigte Objekt verweist.		4		
c) Ergänzung des Klassendiagramms Kernpunkte: Klassenkarte Eilauftrag mit frist, Konstruktor (c) und Methoden; Vererbungspfeil mit hohler Dreiecksspitze zu Auftrag; Klassenkarte Abenteurer mit name, gold, auftrag: Auftrag, Konstruktor und Methoden annehmen / abschliessen; Assoziation Abenteurer – Auftrag als einfache, unbeschriftete Linie ohne Kardinalitäten.	1	2	1	
d) Angabe der Ausgabe; Erläuterung der ausgeführten Methoden Kernpunkte: Ausgabe 80 50 40; für e1 und e2 wird die in Eilauftrag überschriebene Methode ausgeführt, da der Typ des Objekts zur Laufzeit entscheidet (nicht der Variablentyp Auftrag); super.getBelohnung() liefert den in Auftrag gespeicherten Grundwert 40, der verdoppelt (frist ≤ 2) bzw. um 10 erhöht wird; für n1 läuft die Methode aus Auftrag.	1	2		
e) Implementierung der Methode leichtesterOffenerAuftrag Kernpunkte: Korrekter Methodenkopf, Durchlauf über die belegten Felder 0 bis anzahl − 1; Auswahl nur nicht erledigter Aufträge; Minimumsuche über getSchwierigkeit; Rückgabe null ohne Laufzeitfehler, wenn kein offener Auftrag existiert (auch leeres Brett); bei Gleichstand wird durch echten Vergleich (<) der erste geliefert; Brett bleibt unverändert.	1	2	2	
f) Analyse der Implementierung von sucheAuftrag Kernpunkte: Fehler: Start bei 1, Bedingung i <= anzahl, Vergleich mit ==; Auswirkungen: erster Auftrag wird nie gefunden, Zugriff auf auftraege[anzahl] (null → NullPointerException, bei vollem Brett Index 10 außerhalb der Reihung), == vergleicht Referenzen statt Inhalte; korrigierte Implementierung mit i = 0, i < anzahl und equals.	1	2	1	
Summe	7	13	4	**/ 24**

AFB-Verteilung: I 7 BE (29 %) · II 13 BE (54 %) · III 4 BE (17 %) – Abitur-Richtwert 30 / 53 / 17 %. Zeitbudget: 24 BE · 2,5 min = 60 min.

Die gewählten Lösungsansätze und -wege müssen nicht mit den dargestellten Lösungen identisch sein. Fachlich richtige Alternativen sind unter Berücksichtigung der verbindlichen Vorgaben (EH 2025) entsprechend zu bewerten.

Rückmeldung zum Boss-Kampf (fließt nicht in die Note ein)

- Auswertung durch die Lehrkraft oder als Selbst-/Peer-Korrektur nach den Kernpunkten dieses Erwartungshorizonts.
- Papier-Compiler zu e): Handschrift unverändert abgetippt, alle 8 Tests grün beim ersten Versuch → **Papier-Compiler bestanden**; sonst Fehlerlogbuch und Korrektur in der IDE.
- Fallen-Check: In e) und f) sind alle Leer-, Index- und null-Abfragen korrekt (e: null-Abfrage vor bester.getSchwierigkeit(), Grenze anzahl; f: i = 0 und i < anzahl).
- Orientierung (informativ, Abi-Skala): ab 95 % 15 P · 85 % 13 P · 75 % 11 P · 65 % 9 P · 55 % 7 P · 45 % 5 P · 40 % 4 P · 27 % 2 P. Für 24 BE: 23 BE ≈ 15 P, 18 BE ≈ 11 P, 10 BE ≈ 4 P.
- Zusätzlich im Lab: Online-Boss **„Der Reitstall der Heldenschmiede“** (25 min, automatische Auswertung, bester Versuch zählt). Die Ergebnisse sieht die Lehrkraft im Lab unter Kurs → Auswertung (Boss-Ergebnisse).

Musterlösungen, Kernpunkte und typische Fehler

a) Erläuterung des Aufbaus der Klasse Auftrag 4 BE (I 3 · II 1 · III 0)

Musterlösung

Eine Klasse ist der Bauplan für gleichartige Objekte. Sie besteht aus Attributen, Konstruktoren und Methoden.

Die Attribute titel, belohnung, schwierigkeit und erledigt speichern den Zustand eines Auftrag-Objekts. Sie sind mit - (private) gekennzeichnet und damit nur innerhalb der Klasse Auftrag zugreifbar. Von außen kann z. B. erledigt nur über die Methoden istErledigt() gelesen und über erledigen() verändert werden (Datenkapselung, Schutz vor ungültigen Werten wie schwierigkeit = 9).

Die Methoden sind mit + (public) gekennzeichnet und können von außen aufgerufen werden: Die get-Methoden und istErledigt() liefern Attributwerte (sondierend), erledigen() verändert den Zustand (verändernd).

Die mit c gekennzeichneten Konstruktoren werden beim Erzeugen eines Objekts mit new aufgerufen und belegen die Attribute mit Anfangswerten. Auftrag besitzt zwei Konstruktoren mit gleichem Namen, aber unterschiedlicher Parameterliste (Überladen): Auftrag(titel) erzeugt einen Standardauftrag mit belohnung = 10 und schwierigkeit = 1, der zweite Konstruktor übernimmt belohnung und schwierigkeit aus den Parametern (mit Prüfung des Bereichs 1 bis 5). Welcher Konstruktor ausgeführt wird, entscheiden Anzahl und Typen der Argumente.

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Bestandteile Attribut, Konstruktor, Methode mit Bezug zu Auftrag benannt und beschrieben	I	1	☐
Bedeutung von - und + erläutert (Zugriff nur innerhalb / auch von außen)	I	1	☐
Aufgabe der Konstruktoren (c): Objekterzeugung, Initialisierung	I	1	☐
Zwei Konstruktoren: Überladen, Auswahl über Parameterliste, Standardwerte	II	1	☐

Alternativen / Teilpunkte: Die Begriffe „sondierend/verändernd“ sind nicht gefordert. Beschreibt der Prüfling die Konstruktoren nur allgemein, ohne auf die unterschiedliche Parameterliste einzugehen, wird der letzte Kernpunkt nicht vergeben.

Typische Fehler

- c wird als „Klassenmethode“ oder „Kommentar“ gedeutet.
- „private“ wird als „geheim für den Spieler“ statt als Zugriffsbeschränkung auf die Klasse erklärt.
- Konstruktor wird mit Rückgabetyp beschrieben oder mit einer get-Methode verwechselt.

b) Darstellung des Zustands mit Objektkarten; Angabe und Begründung 4 BE (I 0 · II 4 · III 0)

Musterlösung

Siehe Abbildung EH.1. Es entstehen genau drei Auftrag-Objekte. a3 = a1 kopiert nur die Referenz; daher wird dasselbe Objekt zweimal angeheftet (Felder 0 und 2). a3.erledigen() verändert dieses gemeinsame Objekt. Die letzte Zeile lässt a2 auf ein neues Objekt verweisen; das Brett verweist in Feld 1 weiterhin auf „Drachenei bergen“.

brett.getAuftrag(2).istErledigt() liefert wahr: Feld 2 verweist auf dasselbe Objekt wie a1 und a3, und dieses wurde über a3 erledigt.

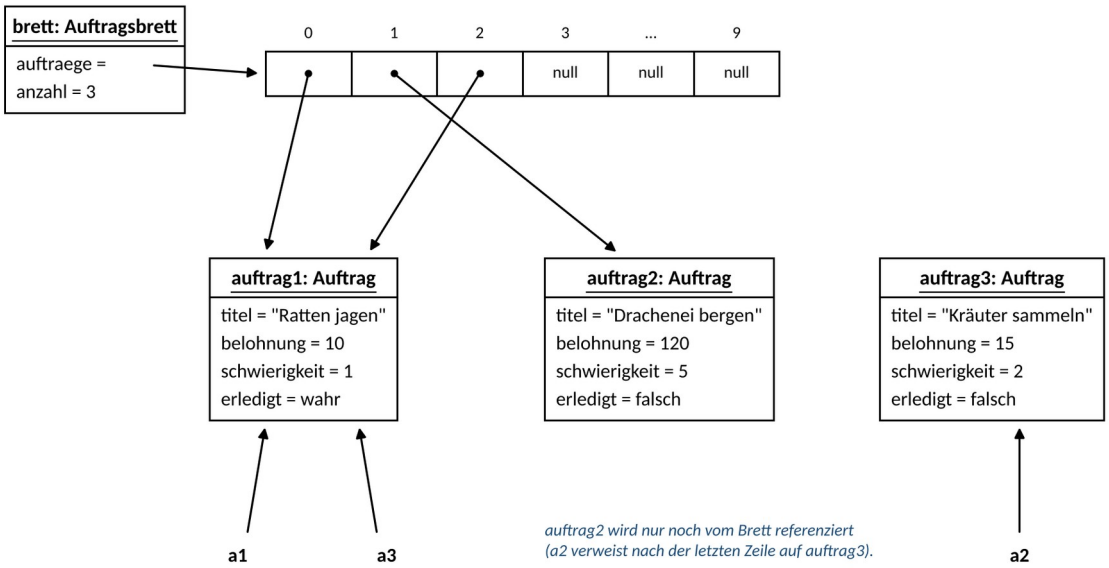


Abbildung EH.1: Objektkarten nach Ausführung von Abbildung 2

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Objektkarten der drei Auftrag-Objekte mit korrekten Attributwerten (inkl. Standardwerte)	II	1	☐
Reihung: Felder 0 und 2 → dasselbe Objekt, Feld 1 → Drachenei, Feld 3 null; anzahl = 3	II	1	☐
Variablen: a1, a3 → gemeinsames Objekt; a2 → Kräuter-Objekt; Drachenei nur noch über das Brett erreichbar	II	1	☐
Wert wahr mit Begründung über die gemeinsame Referenz	II	1	☐

Alternativen / Teilpunkte: Die Objektnamen (auftrag1 …) sind frei wählbar. Die Reihung darf als eigenes Objekt oder als Feldliste innerhalb der Objektkarte brett dargestellt werden. Werden die Felder 4 bis 9 nicht gezeichnet, ist das nicht fehlerhaft.

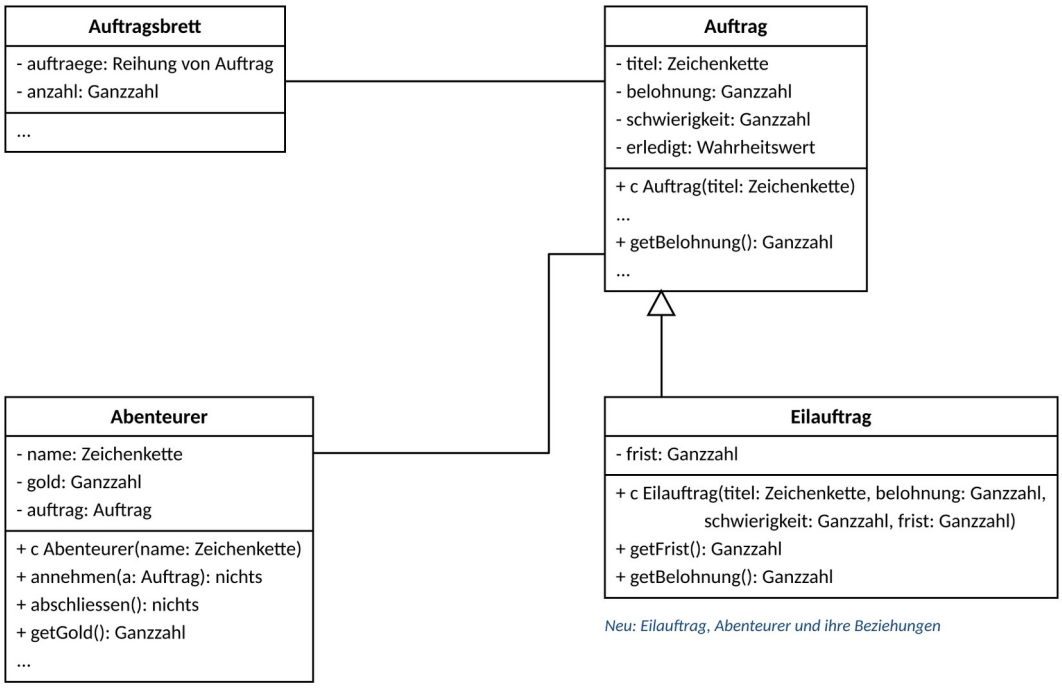
Typische Fehler

- Für a3 wird ein viertes Objekt („Kopie“) gezeichnet.
- Feld 1 wird nach der letzten Zeile auf „Kräuter sammeln“ umgebogen.
- Nur a3 gilt als erledigt, a1 bzw. Feld 0 nicht.
- Standardwerte des Konstruktors mit einem Parameter fehlen (belohnung = 10, schwierigkeit = 1).

c) Ergänzung des Klassendiagramms 4 BE (I 1 · II 2 · III 1)

Musterlösung

Siehe Abbildung EH.2.



Neu: Eilauftrag, Abenteurer und ihre Beziehungen

Abbildung EH.2: ergänztes Klassendiagramm (Auftrag und Auftragsbrett verkürzt)

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Klassenkarte Eilauftrag (nur zusätzliche Attribute, Konstruktor mit c, getFrist, überschriebenes getBelohnung)	I	1	☐
Vererbung Eilauftrag → Auftrag mit hohler Dreiecksspitze an der Oberklasse	II	1	☐
Klassenkarte Abenteurer mit sinnvollen Attributen (Sichtbarkeit, Typnamen) inkl. Referenzattribut auftrag: Auftrag	II	1	☐
Assoziation Abenteurer – Auftrag korrekt; Methoden annehmen/abschliessen mit sinnvollen Signaturen	III	1	☐

Alternativen / Teilpunkte: Weitere sinnvolle Methoden (z. B. getName(), hatAuftrag(): Wahrheitswert) oder annehmen(a: Auftrag): Wahrheitswert sind gleichwertig. Ein Startgold als Konstruktorparameter ist zulässig. Der Rückgabetyp nichts darf – wie in den Abitur-Klassenkarten – auch weggelassen werden.

Typische Fehler

- In Eilauftrag werden titel, belohnung usw. erneut aufgeführt.
- Pfeilspitze zeigt auf die Unterklasse oder ist gefüllt.
- Beschriftete Linien oder Kardinalitäten (im Abitur nicht vorgesehen).
- Abenteurer erhält eine Reihung von Auftrag, obwohl er höchstens einen Auftrag hat.

d) Angabe der Ausgabe; Erläuterung der ausgeführten Methoden 3 BE (I 1 · II 2 · III 0)

Musterlösung

Ausgabe: 80 50 40

e1 und e2 sind Variablen vom Typ Auftrag, verweisen aber auf Eilauftrag-Objekte. Ausgeführt wird die Methode des tatsächlichen Objekttyps, also das in Eilauftrag überschriebene getBelohnung() (dynamische Bindung). Dort liefert super.getBelohnung() den Grundwert 40 aus Auftrag: Bei e1 ist frist = 1 ≤ 2, also 40 · 2 = 80; bei e2 ist frist = 5, also 40 + 10 = 50. n1 ist ein Auftrag-Objekt, daher wird getBelohnung() aus Auftrag ausgeführt: 40.

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Ausgabe 80 50 40	I	1	☐
Laufzeittyp entscheidet: überschriebene Methode bei e1, e2; Methode aus Auftrag bei n1	II	1	☐
Rolle von super.getBelohnung() und Fallunterscheidung über frist	II	1	☐

Alternativen / Teilpunkte: Die Fachbegriffe „dynamische Bindung“ oder „Polymorphie“ sind nicht zwingend, wenn das Prinzip korrekt beschrieben ist. Bei falscher Ausgabe, aber richtiger Erläuterung werden die Kernpunkte zu II vergeben.

Typische Fehler

- Ausgabe 40 40 40 („Variable ist vom Typ Auftrag“).
- super.getBelohnung() wird als rekursiver Aufruf gedeutet.
- Ausgabe ohne Leerzeichen oder untereinander.

e) Implementierung der Methode leichtesterOffenerAuftrag 5 BE (I 1 · II 2 · III 2)

Musterlösung

Musterlösung (Java, mit javac und den Papier-Compiler-Tests geprüft):

```
public Auftrag leichtesterOffenerAuftrag() {
    Auftrag bester = null;
    for (int i = 0; i < anzahl; i++) {
        Auftrag a = auftraege[i];
        if (!a.istErledigt()) {
            if (bester == null
                    || a.getSchwierigkeit() < bester.getSchwierigkeit()) {
                bester = a;
            }
        }
    }
    return bester;
}
```

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Methodenkopf passend zur Signatur; Schleife über 0 bis anzahl − 1	I	1	☐
Nur nicht erledigte Aufträge werden berücksichtigt (!istErledigt())	II	1	☐
Minimumsuche über getSchwierigkeit() mit Merken des besten Auftrags	II	1	☐
Kein offener Auftrag / leeres Brett: null ohne NullPointerException (null-Abfrage vor bester.getSchwierigkeit())	III	1	☐
Gleichstand: erster Auftrag durch < (nicht <=); Brett und Aufträge unverändert	III	1	☐

Alternativen / Teilpunkte: Gleichwertig: Merken eines Index (z. B. bi = -1, Rückgabe null bei bi == -1); zwei Durchläufe (erst ersten offenen Auftrag suchen, dann ab dort das Minimum). Struktogramm statt Java: höchstens 3 BE, da Java gefordert ist. Kleinere Syntaxfehler (fehlendes Semikolon) führen nicht zu Abzug, wenn die Logik eindeutig ist – beim Papier-Compiler zählen sie aber!

Typische Fehler

- bester = auftraege[0] als Startwert – der erste Auftrag kann erledigt sein, bei leerem Brett ist er null.
- Schleife bis auftraege.length → NullPointerException bei weniger als 10 Aufträgen.
- <= statt < → bei Gleichstand wird der letzte geliefert.
- return innerhalb der Schleife (liefert den ersten offenen Auftrag).
- Aufträge werden beim Suchen erledigt oder aus dem Brett entfernt.

f) Analyse der Implementierung von sucheAuftrag 4 BE (I 1 · II 2 · III 1)

Musterlösung

Fehler 1: Die Schleife beginnt bei i = 1. Der Auftrag in Feld 0 wird nie geprüft und daher nie gefunden.

Fehler 2: Die Bedingung i <= anzahl führt im letzten Durchlauf zum Zugriff auf auftraege[anzahl]. Dieses Feld ist null, der Aufruf getTitel() löst eine NullPointerException aus; bei vollem Brett (anzahl = 10) liegt der Index sogar außerhalb der Reihung. Der Fehler tritt immer dann auf, wenn der gesuchte Titel nicht vorher gefunden wird – also auch bei einem nicht vorhandenen Titel, für den null geliefert werden soll.

Fehler 3: == vergleicht bei Zeichenketten die Referenzen, nicht den Inhalt. Inhaltsgleiche, aber verschiedene Zeichenketten-Objekte (z. B. aus einer Eingabe) werden nicht als gleich erkannt. Korrekt ist equals. (Hinweis: Die Online-IDE vergleicht Zeichenketten mit == inhaltlich – der Fehler fällt dort im Test nicht auf. Im Abitur und in Java gilt: equals.)

Korrigierte Implementierung:

```
public Auftrag sucheAuftrag(String titel) {
    for (int i = 0; i < anzahl; i++) {
        if (auftraege[i].getTitel().equals(titel)) {
            return auftraege[i];
        }
    }
    return null;
}
```

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Identifikation der drei Fehler	I	1	☐
Auswirkungen von Fehler 1 und Fehler 2 (erster Auftrag übersehen; Zugriff auf null bzw. außerhalb der Reihung)	II	1	☐
Auswirkung von Fehler 3 (Referenz- statt Inhaltsvergleich)	II	1	☐
Vollständig korrigierte Implementierung	III	1	☐

Alternativen / Teilpunkte: Eine Korrektur, die Fehler 2 durch eine zusätzliche null-Abfrage (auftraege[i] != null) abfängt, ist gleichwertig, sofern Feld 0 geprüft wird. compareTo(…) == 0 statt equals ist ebenfalls korrekt.

Typische Fehler

- Nur zwei Fehler gefunden (meist fehlt == statt equals).
- Auswirkung von Fehler 2 wird mit „findet den letzten nicht“ falsch beschrieben.
- Korrektur ändert return null in die Schleife (else-Zweig).

Anhang: Papier-Compiler-Workspace zu e)

Für den Papier-Compiler legst du in der Online-IDE einen Workspace „W1 Boss 1 – Papier-Compiler“ mit den Klassen Auftrag, Eilauftrag, Auftragsbrett (Methode leichtesterOffenerAuftrag als leerer Rumpf mit return null;) und der Testklasse unten an und verteilst ihn über „An Klasse austeilen“. Die Lernenden tippen ihre Handschrift **unverändert** in den Rumpf ab und starten die Tests im Reiter „Testrunner“. Erst danach dürfen Fehler korrigiert werden (dann ohne Erster-Versuch-Bonus).

Vor Einsatz prüfen

JUnit-Dialekt der Online-IDE (parameterloser Konstruktor der Testklasse, @Test, globale assertEquals/assertTrue/assertFalse mit Meldung als letztem Argument) vor dem ersten Einsatz mit einem Probedurchlauf testen. Die Bibliothek „niedersachsen“ wird hier nicht benötigt.

Klasse Auftragsbrett (Gerüst für den Workspace)

```
public class Auftragsbrett {
    private Auftrag[] auftraege;
    private int anzahl;

    public Auftragsbrett() {
        auftraege = new Auftrag[10];
        anzahl = 0;
    }

    public boolean anheften(Auftrag a) {
        if (a == null || anzahl == auftraege.length) {
            return false;
        }
        auftraege[anzahl] = a;
        anzahl++;
        return true;
    }

    public int getAnzahl() { return anzahl; }

    public Auftrag getAuftrag(int i) {
        if (i < 0 || i >= anzahl) {
            return null;
        }
        return auftraege[i];
    }

    public Auftrag leichtesterOffenerAuftrag() {
        // HIER die Handschrift aus Aufgabe e) UNVERAENDERT abtippen
        return null;
    }
}
```

Testklasse BossTest (8 Tests; mit der Musterlösung alle grün, mit <= statt < zwei Tests rot – geprüft mit javac)

```
public class BossTest {

    public BossTest() {
    }

    @Test
    public void leeresBrettLiefertNull() {
```

```
        Auftragsbrett b = new Auftragsbrett();
        assertTrue(b.leichtesterOffenerAuftrag() == null,
                "Leeres Brett: null erwartet");
    }

    @Test
    public void einzigerOffenerAuftrag() {
        Auftragsbrett b = new Auftragsbrett();
        Auftrag a = new Auftrag("Ratten jagen");
        b.anheften(a);
        assertTrue(b.leichtesterOffenerAuftrag() == a,
                "Ein offener Auftrag muss geliefert werden");
    }

    @Test
    public void kleinsteSchwierigkeitGewinnt() {
        Auftragsbrett b = new Auftragsbrett();
        Auftrag a1 = new Auftrag("Drachenei bergen", 120, 5);
        Auftrag a2 = new Auftrag("Wolf vertreiben", 40, 2);
        Auftrag a3 = new Auftrag("Hexe besuchen", 60, 4);
        b.anheften(a1);
        b.anheften(a2);
        b.anheften(a3);
        assertTrue(b.leichtesterOffenerAuftrag() == a2,
                "Schwierigkeit 2 ist die kleinste");
    }

    @Test
    public void beiGleichstandDerErste() {
        Auftragsbrett b = new Auftragsbrett();
        Auftrag a1 = new Auftrag("Kraeuter sammeln", 15, 2);
        Auftrag a2 = new Auftrag("Wolf vertreiben", 40, 2);
        b.anheften(a1);
        b.anheften(a2);
        assertTrue(b.leichtesterOffenerAuftrag() == a1,
                "Bei Gleichstand muss der erste geliefert werden");
    }

    @Test
    public void erledigteWerdenIgnoriert() {
        Auftragsbrett b = new Auftragsbrett();
        Auftrag a1 = new Auftrag("Ratten jagen");
        Auftrag a2 = new Auftrag("Drachenei bergen", 120, 5);
        b.anheften(a1);
        b.anheften(a2);
        a1.erledigen();
        assertTrue(b.leichtesterOffenerAuftrag() == a2,
                "Erledigte Auftraege duerfen nicht geliefert werden");
    }

    @Test
    public void alleErledigtLiefertNull() {
        Auftragsbrett b = new Auftragsbrett();
        Auftrag a1 = new Auftrag("Ratten jagen");
        b.anheften(a1);
        a1.erledigen();
        assertTrue(b.leichtesterOffenerAuftrag() == null,
                "Kein offener Auftrag: null erwartet");
    }
```

```
    @Test
    public void vollesBrett() {
        Auftragsbrett b = new Auftragsbrett();
        for (int i = 0; i < 10; i++) {
            b.anheften(new Auftrag("Auftrag " + i, 10, 5 - i % 5));
        }
        assertEquals(1, b.leichtesterOffenerAuftrag().getSchwierigkeit(),
                "Volles Brett: Schwierigkeit 1 erwartet");
        assertEquals("Auftrag 4", b.leichtesterOffenerAuftrag().getTitel(),
                "Volles Brett: erster Auftrag mit Schwierigkeit 1");
    }

    @Test
    public void brettBleibtUnveraendert() {
        Auftragsbrett b = new Auftragsbrett();
        Auftrag a1 = new Auftrag("Ratten jagen");
        Auftrag a2 = new Auftrag("Wolf vertreiben", 40, 2);
        b.anheften(a1);
        b.anheften(a2);
        b.leichtesterOffenerAuftrag();
        assertEquals(2, b.getAnzahl(), "anzahl darf sich nicht aendern");
        assertTrue(b.getAuftrag(0) == a1 && b.getAuftrag(1) == a2,
                "Reihenfolge darf sich nicht aendern");
        assertFalse(a1.istErledigt(), "Auftraege duerfen nicht erledigt werden");
    }
}
```

Die Klassen Auftrag und Eilauftrag entsprechen Abbildung 1 und 3 (Auftrag(titel) leitet mit this(titel, 10, 1) an den zweiten Konstruktor weiter).

Klasse Auftrag (für den Workspace)

```
public class Auftrag {
    private String titel;
    private int belohnung;
    private int schwierigkeit;
    private boolean erledigt;

    public Auftrag(String titel) {
        this(titel, 10, 1);
    }

    public Auftrag(String titel, int belohnung, int schwierigkeit) {
        this.titel = titel;
        this.belohnung = belohnung;
        if (schwierigkeit >= 1 && schwierigkeit <= 5) {
            this.schwierigkeit = schwierigkeit;
        } else {
            this.schwierigkeit = 1;
        }
        erledigt = false;
    }

    public String getTitel() { return titel; }
    public int getBelohnung() { return belohnung; }
    public int getSchwierigkeit() { return schwierigkeit; }
    public boolean istErledigt() { return erledigt; }
    public void erledigen() { erledigt = true; }
}
```